

A Technical Report on Blockchain Configuration Optimization



M.Tech CS

Semester : II

Subject: Evolutionary Computation

Submitted by : Kavita Pat Pingua

Roll No : 2025PGCSCS20

Submitted to : Dr. Dinesh Kumar

PERFORMANCE EVALUATION OF META-HEURISTICS TECHNIQUES FOR BLOCKCHAIN CONFIGURATION OPTIMIZATION (BCO): A COMPARATIVE STUDY	3
ABSTRACT	3
1. INTRODUCTION.....	3
1.1 OVERVIEW	3
1.2 IMPORTANCE AND COMPLEXITY OF THE BCO	3
1.3 ROLE OF META-HEURISTIC TECHNIQUES	3
1.4 OBJECTIVE	3
2. BACKGROUND	3
2.1 BLOCKCHAIN CONFIGURATION OPTIMIZATION (BCO)	4
2.1 META-HEURISTICS OVERVIEW	5
3. PROBLEM DEFINITION	5
4. COMPARATIVE STUDY OF METAHEURISTIC TECHNIQUES.....	6
4.1 BASIC WORKING PRINCIPLE OF ALGORITHMS	6
4.1.1 BCGA :.....	6
4.1.2 RCGA :.....	6
4.1.3 Hybrid (BCGA + RCGA) :.....	6
4.1.4 PSO :	7
4.1.5 TLBO :	7
4.1.6 DE :	7
4.1.7 ABC :	7
5. EXPERIMENTAL ANALYSIS.....	7
5.1 PARAMETER SETTINGS.....	7
5.2 RESULTS AND DISCUSSION.....	9
5.3 DISCUSSION	16
6. CONCLUSION	16

Performance Evaluation of Meta-heuristics Techniques for Blockchain Configuration Optimization (BCO): A Comparative Study

Abstract

In this work, the main emphasis is on Blockchain Configuration Optimization (BCO). It aims at enhancing the performance and efficiency of a blockchain in different cases. Since the issue is quite complicated, different metaheuristic algorithms have been tested, among which we can mention Genetic Algorithms (BCGA, RCGA), Particle Swarm Optimization (PSO), Teaching-Learning-Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC).

These algorithms have been checked in various experiments involving changing parameters of the blockchain, such as the amount of transactions and size of blocks, according to their convergence rate and efficiency. The fastest convergence was reached by PSO and DE, with TLBO and ABC providing comparable results but being more straightforward to implement. Meanwhile, Genetic Algorithms were reliable yet relatively slow.

Metaheuristic algorithms proved to be useful in solving the problem of BCO.

1. Introduction

1.1 Overview

Combinatorial optimization is the process of identifying the best combination among various possible solutions. Combinatorial optimization is usually complicated and resource-intensive.

1.2 Importance and complexity of the BCO

Blockchain Configuration Optimization (BCO) is an example of a combinatorial optimization problem. This type of problem concerns choosing suitable configurations that can enhance performance, scalability, and efficiency. The BCO problem is difficult to solve owing to its non-linear and dynamic characteristics.

1.3 Role of meta-heuristic techniques

Meta-heuristics are popular methods for addressing combinatorial optimization issues. Meta-heuristics yield solutions that are close to optimal within acceptable time periods. Meta-heuristics such as genetic algorithms, PSO, TLBO, DE, and ABC can be effective in managing extensive search spaces.

1.4 Objective

This research aims to evaluate several metaheuristic algorithms in the context of the BCO problem.

2. Background

2.1 Blockchain Configuration Optimization (BCO)

In Blockchain networks for healthcare applications, patients' transactions are packed into blocks and validated through the cooperation of a set of validator nodes. There are two primary control parameters in the system, which affect the system's performance: first, the validator set $\mathcal{M}$ (of size m), drawn from the set of all available validators M , and second, the number of transactions per block n . These parameters form a conflicting objective function optimization problem: adding more validators enhances security at the expense of higher cost and delay, whereas packing more transactions per block enhances performance but increases validation delay.

Formal Problem Definition:

The BCO problem is formally stated as:

$$P: \min_{\mathcal{M}, n} U$$

Where

$$U = \alpha \cdot L_{norm} + \beta \cdot \eta_{norm} + \gamma \cdot C_{norm}$$

- L_{norm} : normalized block verification latency
- η_{norm} : normalized security level
- C_{norm} : normalized cost
- α, β, γ : weighting coefficients ($\alpha + \beta + \gamma = 1$)
- For normalization, we need to use log min-max function:

$$\text{Norm value} = \frac{\lg(\text{actual value}) - \lg(\text{min value})}{\lg(\text{max value}) - \lg(\text{min value})}$$

*For normalized security level (As this objective is maximization)

$$\eta_{norm} = 1 - \text{Norm value}$$

Constraints

1. Validator bounds

$$v \leq m \leq M$$

2. Transactions per block bounds

$$t \leq n \leq \mathcal{X}$$

Supporting Definitions

- **Cost function**

$$C = \frac{\sum_{i \in \mathcal{M}} x_i}{n}$$

- Security level

$$\eta = \theta \cdot m^q$$

- Latency (in ms)

$$L = \frac{n \cdot B}{r_d} + \max_{i \in \{v, \dots, M\}} \left(\frac{K}{x_i} \right) + \psi(n \cdot B)m + \frac{0}{r_u}$$

Equations to find min max values:

$$C_{max} = \frac{\sum_{i=1}^M x_i}{t}$$

$$C_{min} = \frac{\sum_{i=1}^v x_i}{X} \text{ (sum of } v \text{ numbers of minimum } x_i \text{ s)}$$

$$\eta_{max} = \theta \cdot (M)^q$$

$$\eta_{min} = \theta \cdot (v)^q$$

$$L_{max} = \frac{X \cdot B}{r_d} + \frac{K}{\min(x_i)} + \psi(X \cdot B)M + \frac{0}{r_u}$$

$$L_{min} = \frac{t \cdot B}{r_d} + \frac{K}{v^{th} \text{ highest } x_i} + \psi(t \cdot B)v + \frac{0}{r_u}$$

2.1 Meta-heuristics Overview

Meta-heuristics represent advanced, problem-universal techniques aimed at solving optimization tasks in vast search spaces. They offer no guarantees of global optimality but can find good solutions relatively quickly. The most representative metaheuristics include evolutionary algorithms (GA, DE), swarm intelligence approaches (PSO, ACO, ABC), and learning-inspired algorithms (TLBO). Although based on different ideas, all metaheuristics feature a unified algorithmic structure, namely the population-based iterative optimization through the alternation of exploration and exploitation.

The BCO problem contains two search components: choosing a subset $\mathcal{M}$ of validators from M total validators and picking an integer number n . Hence, it is NP-hard due to a mixed-integer search space. In particular, the number of ways to choose validators out of 40 is over 2^{40} .

The constraints $v \leq m \leq M$ and $t \leq n \leq X$ mean that feasibility must be managed actively during the search. The conventional operators might produce feasible solutions in terms of the number of validators and indices. Repair techniques, penalties, and encoding schemes specific to the problem could all prove helpful in overcoming feasibility issues. It is essential not to overdo corrections, as too much could lead to premature convergence.

3. Problem Definition

The BCO problem has been formally introduced in Section 2.1. This section consolidates all notation and variable definitions used throughout the study for reference.

Decision Variables:

Symbol	Description
$\mathcal{M}$	Set of selected validators, $ \mathcal{M} = m$
m	Number of selected validators
n	Number of transactions per block

Objective and Weighting:

Symbol	Description
U	Overall utility function (minimized)
α, β, γ	Weights for latency, security, and cost ($\alpha + \beta + \gamma = 1$)
L_{norm}	Normalized block verification latency
η_{norm}	Normalized security level
C_{norm}	Normalized validator cost

4. Comparative Study of Meta-heuristic Techniques

4.1 Basic Working Principle of Algorithms

4.1.1 BCGA :

Uses a real-valued chromosome of length M where genes > 0.5 represent a validator is selected. Parents are selected for each generation using tournament selection ($k=3$) and then combined using blend crossover (BLX) and randomly mutated with uniform mutation. A repair function ensures that the chosen number of validators is in $[v, M]$.

4.1.2 RCGA :

Same chromosome structure than BCGA but with more sophisticated operators: Simulated Binary Crossover (SBX) with $\eta=15$ to generate children close to their parents, and Polynomial Mutation with $\eta=20$ for fine-grained local perturbation. The tournament selection is used the same as BCGA.

4.1.3 Hybrid (BCGA + RCGA) :

This is a hybrid algorithm that employs two stages in which early stages employ BCGA while subsequent stages employ RCGA for refinement.

It takes place in two phases within the same population. The operators BCGA (blend crossover + uniform mutation) are applied for broad global exploration in generations 1-150. From generation 151-300, it uses RCGA operators (SBX+polynomial mutation) for refined local exploitation. Population transitions between phases smoothly.

4.1.4 PSO :

Each particle has a position (chromosome) and velocity . Velocity is updated each generation with three components: inertia (w , linearly decaying from 0.9->0.4), cognitive pull towards the particle's personal best ($C1=1.5$), and social pull towards the global best ($C2=1.5$). Position is updated by adding velocity, then clamped to $[0, 1]$ and fixed.

4.1.5 TLBO :

Two phases per generation. In the teacher phase, each learner is pulled towards the teacher (best solution) scaled by a random teaching factor $TF \in \{1,2\}$. In learner phase, every solution selects another solution randomly. If the other solution is better, the solution moves away from it, otherwise, it moves towards it. Greedy acceptance will only keep the updates if they improve fitness.

TLBO is ideal for BCO due to its simplicity and parameter free nature.

4.1.6 DE :

For each target vector, three different members of the population are randomly chosen (a, b, c). Create a mutant vector as $a + F \cdot (b - c)$ with $F=0.8$. The binomial crossover ($CR=0.8$) combines the mutant with the target, always taking at least one gene (j_{rand}) from the mutant. The target is replaced by the trial only if it is equal or better (greedy selection).

DE is an excellent algorithm for BCO because of its powerful search capabilities and great ability to converge on solutions.

4.1.7 ABC :

The size of the food source population is $POP_SIZE/2$. Employed bees generate neighbours by exchanging one random gene with a random partner source. Onlooker bees select food sources in proportion to the inverse of their fitness and do the same. If a source did not improve after $SCOUT_LIMIT=20$ trials, then a scout bee would leave the source and generate a new random solution, avoiding stagnation.

ABC is appropriate for BCO since it maintains diversity and does not get trapped in local optima.

5. Experimental Analysis

5.1 Parameter Settings

- Population size: 150
- Number of generations: 300
- Number of runs: 20
- Convergence tolerance: $1e-6$
- Crossover rate: 0.8
- Mutation rate: 0.1
- Selection method: Tournament selection ($k=3$)
- BCGA crossover: Blend crossover (BLX-0.5)
- BCGA mutation: Uniform mutation
- RCGA crossover: Simulated Binary Crossover (SBX, $\eta_c=15$)

- RCGA mutation: Polynomial mutation ($\eta_m=20$)
- Hybrid phase switch: Generation 150 (BCGA-> RCGA)
- PSO: $w = 0.9$, $c1=c2 = 1.5$, $V_{max}=0.4$
- DE: $F = 0.8$, $CR = 0.8$
- ABC: employed/onlooker bees = 75, scout limit = 20
- TLBO: TF – randomly chosen from {1,2}

```
POP_SIZE = 150      # Population / swarm size (100-200 range)
MAX_GEN  = 300      # Maximum generations
RUNS     = 20       # Independent runs
CR       = 0.8      # Crossover rate
MUT_RATE = 0.1      # Mutation rate

CONVERGENCE_TOL = 1e-6 # Tolerance to detect convergence
CONVERGENCE_ITERS = 20 # Stop if no improvement for this many generations
```

```
q      = 4
theta  = 1
O      = 0.5        # Mb - verification feedback size
rd     = 1.2        # Mb/s - downlink rate
ru     = 1.3        # Mb/s - uplink rate
K      = 100        # computational resources for block verification
B      = 0.5/1024    # Mb - block size (0.5 kb converted to Mb)
psi    = 0.001

# Weighting coefficients (alpha + beta + gamma = 1)
ALPHA  = 1/3
BETA   = 1/3
GAMMA  = 1/3
```

```
# Setting 1: M = 25 validators
S1 = {
    'name' : 'Setting 1 (M=25)',
    'v'    : 5,
    'M'    : 25,
    't'    : 50,
    'X'    : 500,
    'xi'   : np.array([250.74, 187.36, 138.91, 245.20, 271.70,
                       276.80, 119.32, 194.45, 213.02, 138.59,
                       207.21, 215.94, 236.32, 203.08, 145.41,
                       264.83, 286.12, 123.30, 242.89, 121.47,
                       198.76, 254.19, 167.43, 289.05, 132.88])
}

# Setting 2: M = 40 validators
S2 = {
    'name' : 'Setting 2 (M=40)',
    'v'    : 5,
    'M'    : 40,
```



```

't'      : 50,
'X'      : 500,
'xi'     : np.array([121.12, 259.89, 198.40, 234.00, 272.30,
                    134.32, 157.55, 288.65, 210.58, 255.98,
                    249.03, 201.28, 298.93, 157.16, 125.26,
                    167.22, 261.86, 211.60, 184.22, 153.01,
                    116.73, 241.46, 201.56, 290.00, 218.31,
                    232.87, 104.61, 268.79, 185.63, 115.13,
                    236.16, 102.55, 259.15, 226.24, 100.15,
                    142.94, 199.64, 221.28, 150.30, 104.61])
}

# Setting 3: M = 60 validators (generated with fixed seed for reproducibility)
rng_s3 = np.random.default_rng(seed=0)
S3 = {
    'name' : 'Setting 3 (M=60)',
    'v'    : 5,
    'M'    : 60,
    't'    : 50,
    'X'    : 500,
    'xi'   : np.round(rng_s3.uniform(100, 300, 60), 2)
}

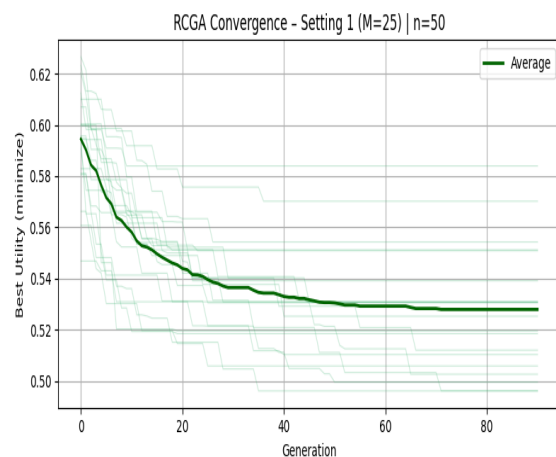
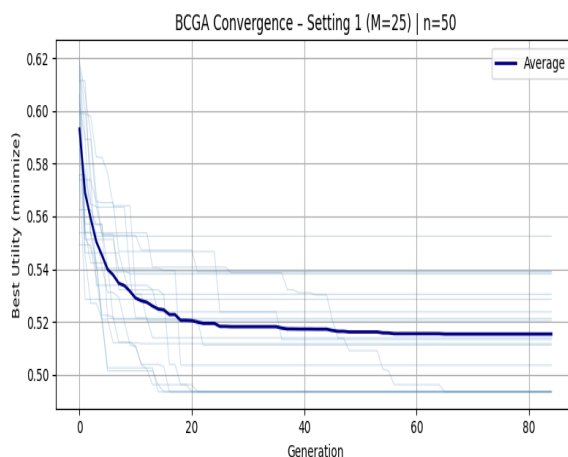
```

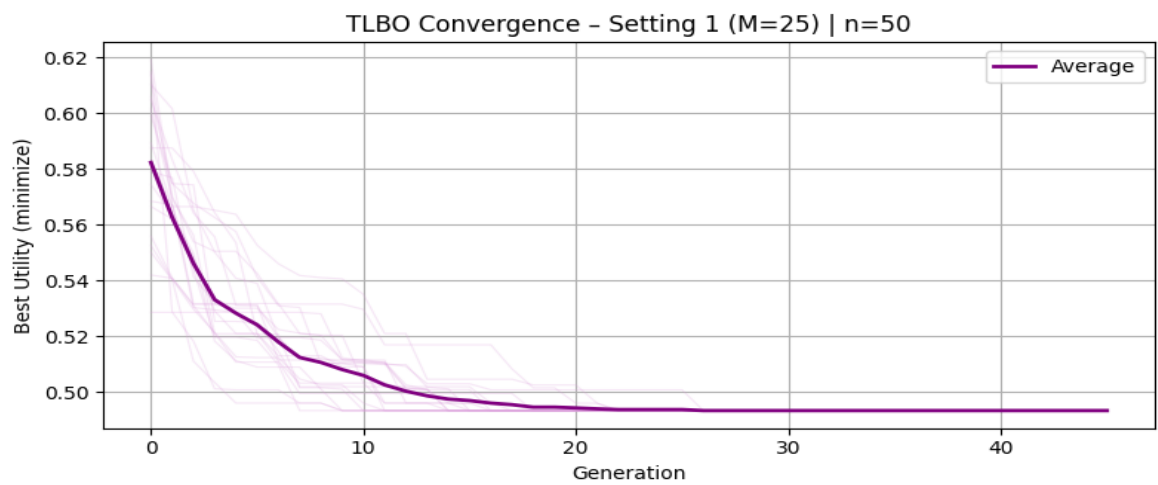
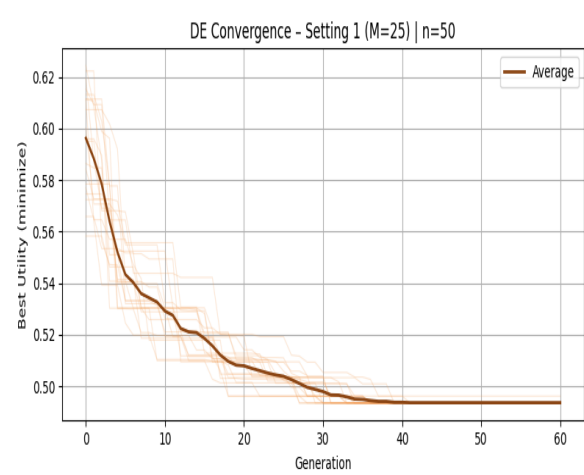
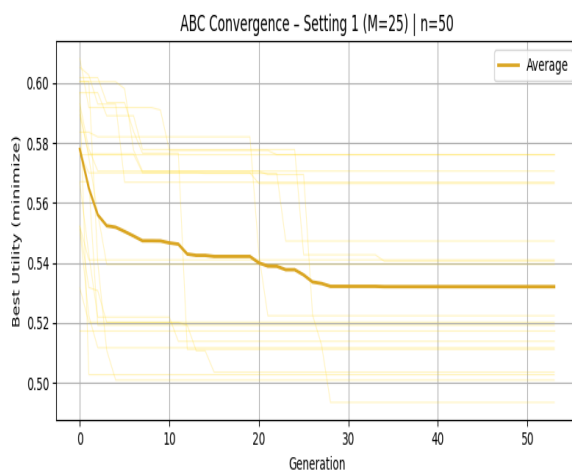
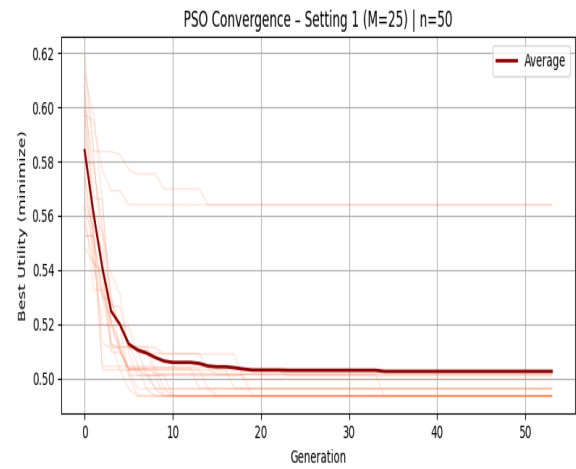
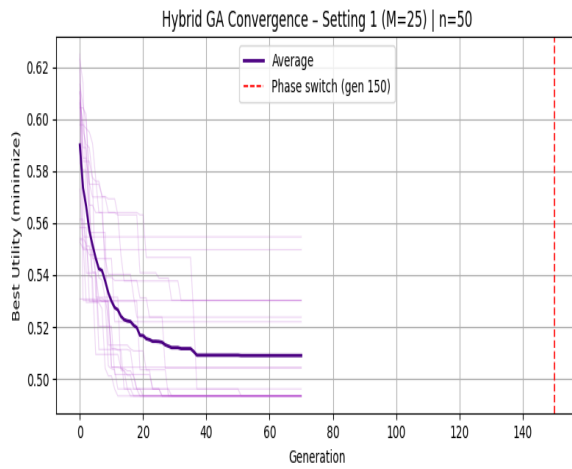
```
TRANSACTION_VALUES = [20, 40, 60, 80, 100]
```

5.2 Results and Discussion

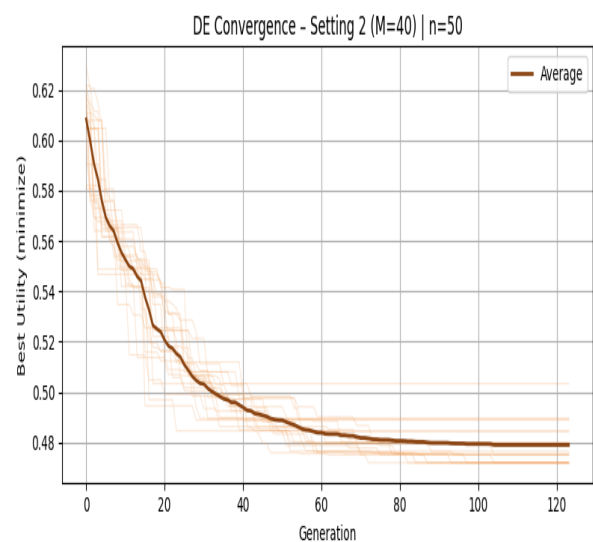
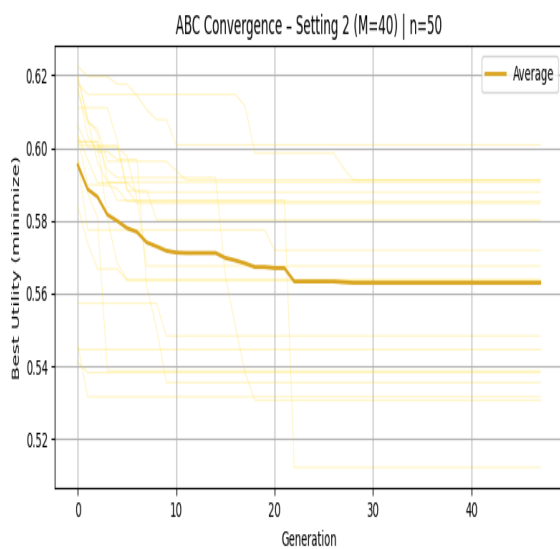
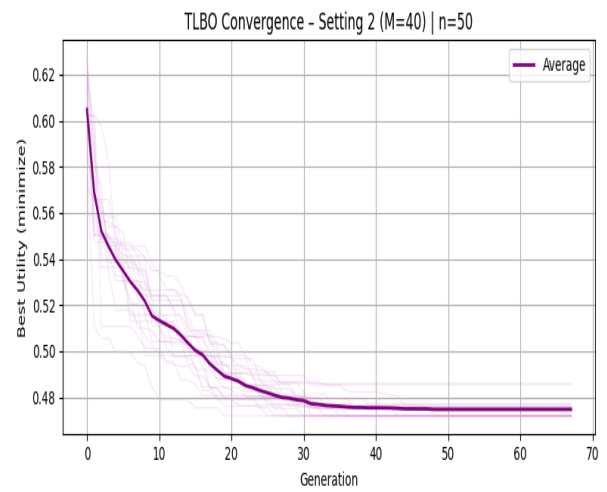
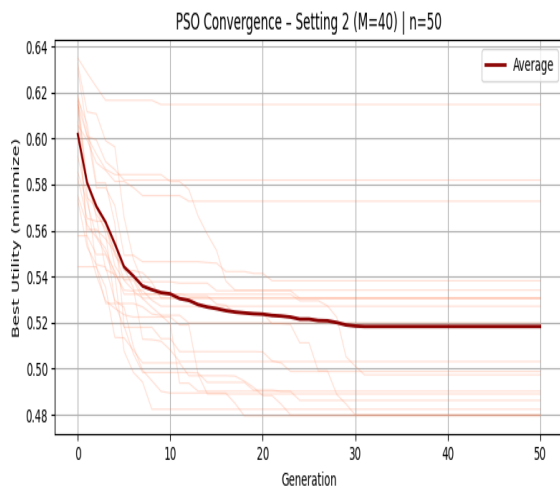
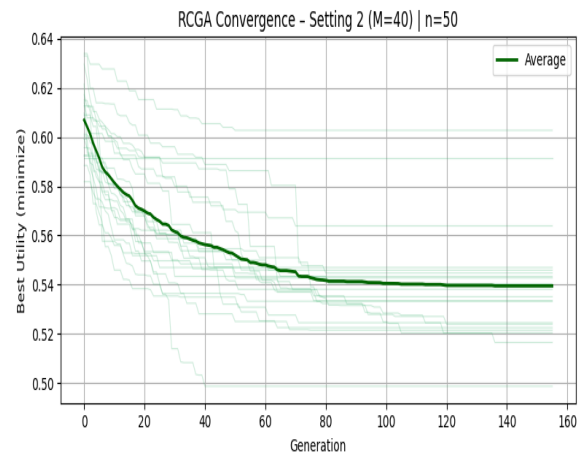
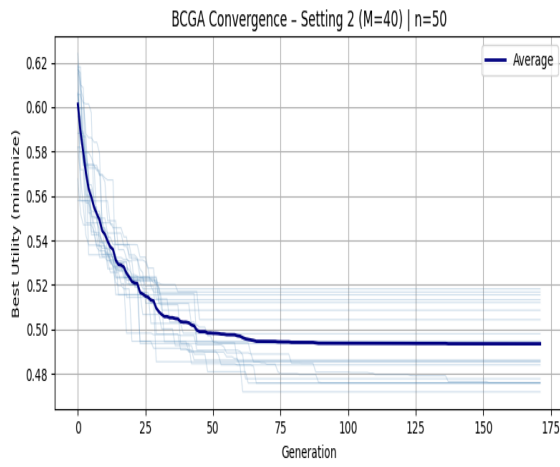
- Graphs and Tables:
 - Convergence curves (Best Utility vs Generations)

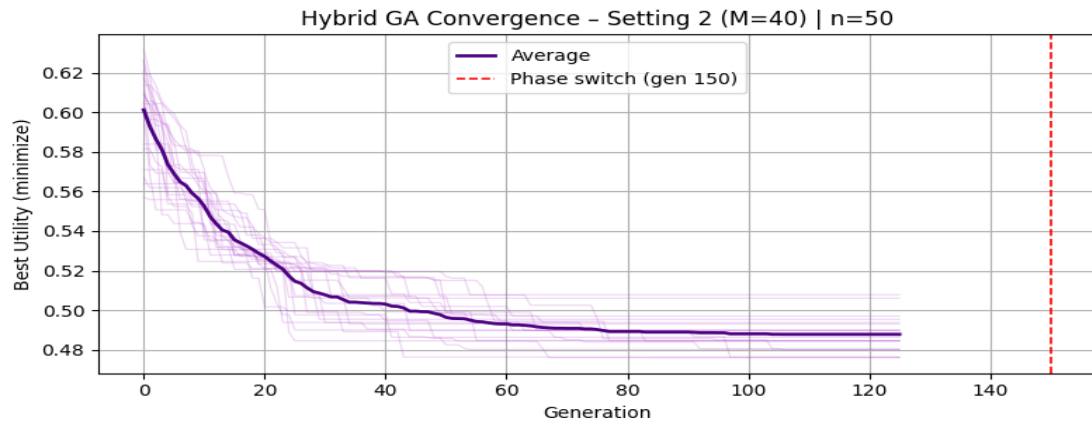
//For Setting 1:



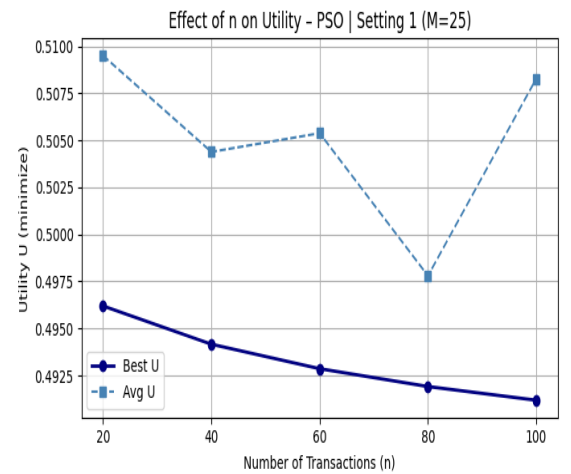
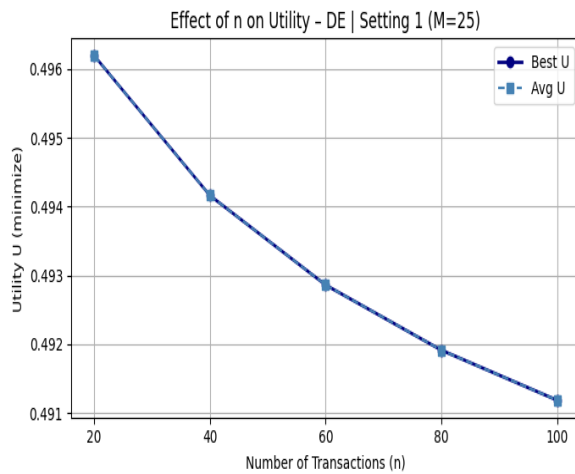
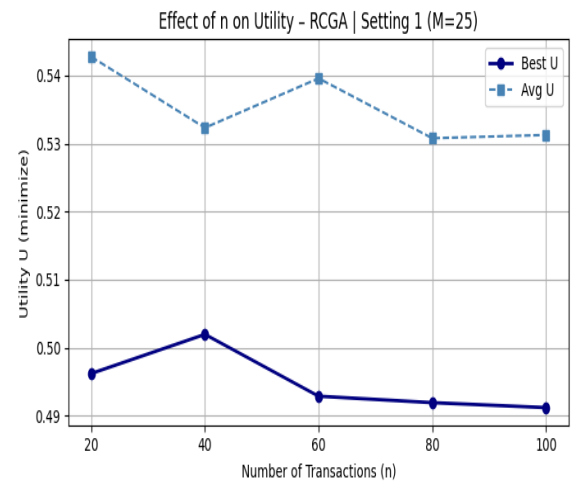
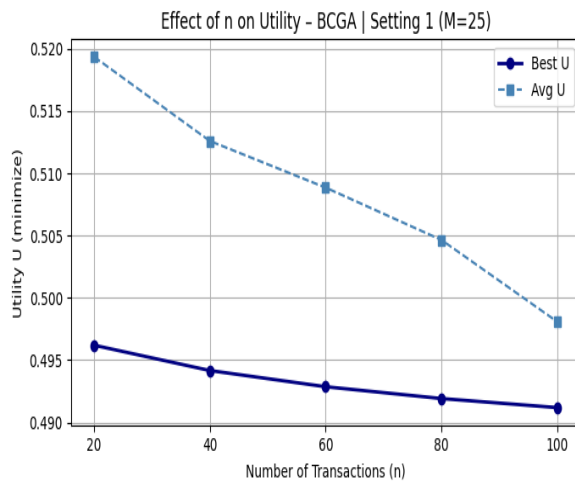


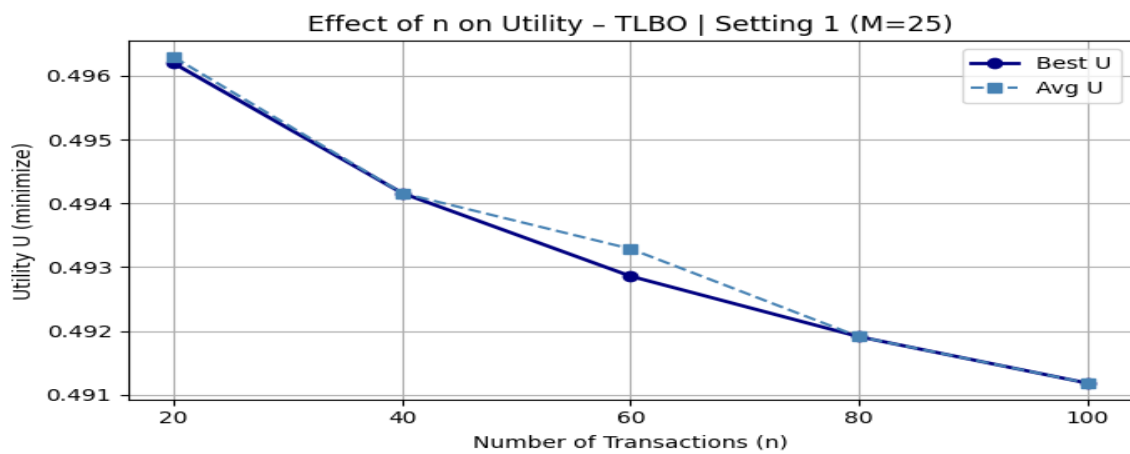
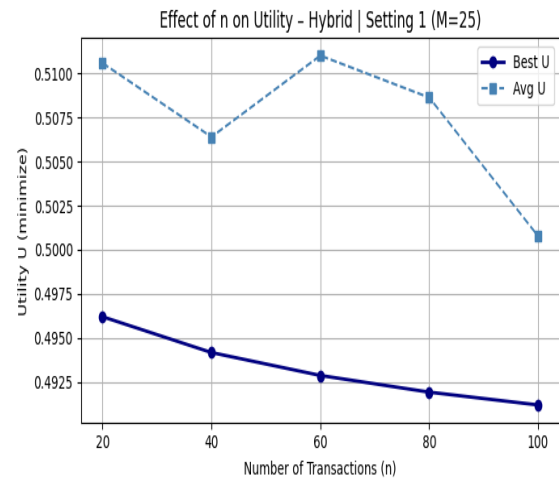
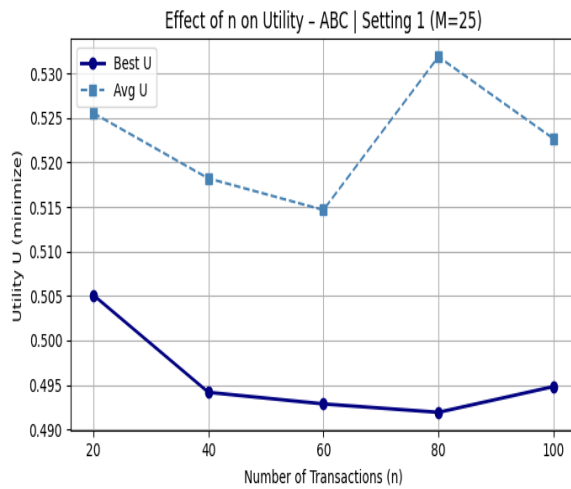
//For Setting 2:





//For different transactions : [20, 40, 60, 80, 100]





➤ Best Solution Found:

COMPARISON TABLE Setting 1 (M=25) n=50					
Algorithm	Best U	Avg U	Std U	Avg Time(ms)	Avg Gen
BCGA	0.493452	0.515359	0.018162	245.59	36.5
RCGA	0.496105	0.527922	0.024008	540.23	53.2
Hybrid	0.493452	0.509005	0.020134	267.31	39.5
PSO	0.493452	0.502533	0.020698	336.12	32.4
TLBO	0.493452	0.493452	0.000000	999.81	34.6
DE	0.493452	0.493586	0.000586	813.47	51.8
ABC	0.493452	0.532061	0.026626	374.23	30.4

COMPARISON TABLE Setting 2 (M=40) n=50					
Algorithm	Best U	Avg U	Std U	Avg Time(ms)	Avg Gen
BCGA	0.471976	0.493569	0.015748	876.99	68.8
RCGA	0.498651	0.539498	0.023643	1532.38	85.8
Hybrid	0.475909	0.487776	0.008891	1126.15	76.2
PSO	0.479580	0.518378	0.036696	783.35	36.0
TLBO	0.471976	0.474761	0.004133	1365.22	52.0
DE	0.471976	0.479049	0.008314	1147.15	85.5
ABC	0.512121	0.562927	0.025414	535.03	28.9

COMPARISON TABLE Setting 3 (M=60) n=50					
Algorithm	Best U	Avg U	Std U	Avg Time(ms)	Avg Gen
BCGA	0.505564	0.516318	0.012041	1157.31	77.9
RCGA	0.525298	0.553801	0.024247	2150.29	99.8
Hybrid	0.507870	0.517610	0.011257	779.18	60.2
PSO	0.504638	0.535602	0.027506	429.95	44.0
TLBO	0.461278	0.468612	0.012562	1691.90	69.2
DE	0.473658	0.497028	0.014576	1457.09	110.5
ABC	0.558447	0.588904	0.015087	581.63	34.4

```
#####
PART A: Main Experiment (n = 50 fixed for all settings)
#####

#####
SETTING: Setting 1 (M=25) (M=25 validators, n=50)
#####

=====
BCGA | Setting 1 (M=25) | n=50
=====
Converged at gen 34

Best solution found (Run 1) - BCGA | Setting 1 (M=25) | n=50:
Utility U      = 0.493452
Validators m    = 16
Transactions n  = 50
Selected xi vals = [250.74, 245.2, 271.7, 276.8, 194.45, 213.02, 207.21, 215.94, 236.32, 203.08, 264.83, 286.12, 242.89,
198.76, 254.19, 289.05]
Selected indices = [0, 3, 4, 5, 7, 8, 10, 11, 12, 13, 15, 16, 18, 20, 21, 23]

BCGA | Setting 1 (M=25) | n=50 - Summary over 20 runs:
Best Utility = 0.493452
Avg Utility  = 0.515359
Worst Utility = 0.552457
Std Utility  = 0.018162
Avg Time     = 245.59 ms
Avg Conv Gen = 36.5
```

//Best Utility value for Setting 1 (M=25)

	BCGA	RCGA	Hybrid	PSO	TLBO	DE	ABC
n = 20	0.496193	0.496193	0.496193	0.496193	0.496193	0.496193	0.505083
n = 40	0.494160	0.501947	0.494160	0.494160	0.494160	0.494160	0.494160

n = 60	0.492859	0.492859	0.492859	0.492859	0.492859	0.492859	0.492859
n = 80	0.491912	0.491912	0.491912	0.491912	0.491912	0.491912	0.491912
n = 100	0.491185	0.491185	0.491185	0.491185	0.491185	0.491185	0.494808

//Best Utility value for Setting 2 (M=40)

	BCGA	RCGA	Hybrid	PSO	TLBO	DE	ABC
n = 20	0.479165	0.497443	0.475053	0.482841	0.475053	0.475053	0.526360
n = 40	0.472790	0.490361	0.479880	0.472790	0.472790	0.472790	0.524691
n = 60	0.474964	0.483467	0.474568	0.471278	0.471278	0.471278	0.517643
n = 80	0.470120	0.486570	0.472939	0.474088	0.470120	0.470120	0.527197
n = 100	0.469180	0.486755	0.475561	0.475261	0.469180	0.469180	0.518854

//Best Utility value for Setting 3 (M=60)

	BCGA	RCGA	Hybrid	PSO	TLBO	DE	ABC
n = 20	0.501953	0.503985	0.508264	0.503995	0.465514	0.479373	0.548106
n = 40	0.508872	0.506499	0.505081	0.508100	0.462411	0.471275	0.531324
n = 60	0.503027	0.504274	0.504510	0.481037	0.460300	0.471005	0.531204
n = 80	0.504585	0.513356	0.506173	0.495699	0.458653	0.463223	0.530214
n = 100	0.500435	0.511083	0.496769	0.493045	0.457257	0.459449	0.537725

Analysis of best utility values across transaction values (n=20,40,60,80,100)

- **Setting 1:** Almost identical across all algorithms

All algorithms achieve roughly the same best utility value at each transaction level. This is because Setting 1 has the smallest search space (M=25 validators), which is easy enough for all algorithms to find the same global optimum, independent of their search strategy. The only exceptions are ABC at n=20 (0.505083) and n=100 (0.494808) where ABC performs slightly worse - in line with the known weakness of ABC to stagnate early on simple landscapes.

As n increases from 20 to 100, the best utility of all algorithms decreases gradually from ~0.496 to ~0.491. This is physically intuitive: The more transactions per block, the lower the cost per transaction $C = \sum \frac{x_i}{n}$, which means lower overall utility U.

- **Setting 2:** Algorithms begin to diverge

A larger validator pool leads to a significantly larger search space (2^{40} combinations) and algorithmic differences begin to emerge. RCGA always gets the worst best utility over all n values, unable to escape from local optima. In general ABC is still the worst performer, with best utilities around 0.517-0.527, far from the optimum. Both TLBO and DE are reliable and consistently able to find the true best values together. Again, the same decreasing trend with increasing n holds here -- lower utility with higher n.

- **Setting 3:**

This is where the task truly gets tough (search space of 2^{60}), and when it comes to distinguishing among algorithms, TLBO clearly comes out ahead, with the lowest best utility for each transaction amount, starting from 0.465514 at n=20 and gradually dropping to 0.457257 at n=100. No other algorithm can match this level of consistency.

The next best algorithm is DE, which also shows a decrease with increasing n . The BCGA, RCGA, Hybrid GA, and PSO algorithms all stagnate near the utility range of 0.500-0.513, implying that they get caught in local optima on this more difficult terrain.

Lastly, ABC proves to be the worst-performing algorithm at all transaction levels and all n values in Setting 3, with its best utility hovering around 0.530-0.548, almost 0.08-0.09 away from the best utility of TLBO.

➤ Average utility comparison

- TLBO, DE → highest and stable average
- RCGA, Hybrid → competitive
- PSO, BCGA → moderate
- ABC → lowest

5.3 Discussion

Seven metaheuristic algorithms – BCGA, RCGA, Hybrid GA, PSO, TLBO, DE, and ABC - were analyzed for the purpose of solving the Blockchain Configuration Optimization (BCO) problem under three different conditions, starting with the easiest one with $M = 25$ and ending with $M = 60$, as well as across the five levels of transactions' values, ranging from $n = 20$ to $n = 100$.

It is evident that the scale of the problem has great influence on the result attained. Under Setting 1 ($M = 25$), all the algorithms yield the same optimal utility. As soon as M reaches the levels seen in Settings 2 and 3, the difference becomes apparent. TLBO manages to deliver the best utility values at each setting, and especially in Setting 3, when its teacher-learner approach proves to be useful in exploring the big solution space. Second place belongs to DE, with RCGA and ABC lagging significantly behind: the former algorithm fails due to crossover inefficiency with respect to binary-threshold encoding, whereas the latter experiences stagnation very fast.

In addition, the increase of the number of transactions n from 20 to 100 consistently leads to a decrease in the utility value obtained.

Encoding and Decoding Table:

Algorithm	Encoding	Decoding
BCGA	Real-valued chromosome of length M . Each gene $\in [0,1]$ represents one validator	Gene $> 0.5 \rightarrow$ validator selected; Gene $\leq 0.5 \rightarrow$ not selected. n is fixed at 50
RCGA	Real-valued chromosome of length M . Each gene $\in [0,1]$ represents one validator	Same threshold decoding as BCGA. SBX and polynomial mutation operate directly on real values before decoding
Hybrid GA	Real-valued chromosome of length M . Identical to BCGA and RCGA encoding	Same threshold decoding. BCGA operators applied for gen 1–150, RCGA operators for gen 151–300
PSO	Each particle is a real-valued position vector of length M . Each dimension $\in [0,1]$	Gene $> 0.5 \rightarrow$ validator selected. Velocity vector of same length controls movement in search space
TLBO	Real-valued learner vector of length M . Each element $\in [0,1]$	Gene $> 0.5 \rightarrow$ validator selected. Teacher is the learner with lowest utility in current population

	represents a validator gene	
DE	Real-valued target vector of length M. Each gene $\in [0,1]$	Trial vector decoded with same 0.5 threshold. Mutant vector formed as $a + F \times (b - c)$ before crossover and decoding
ABC	Each food source is a real-valued vector of length M. Each gene $\in [0,1]$	Gene $> 0.5 \rightarrow$ validator selected. Neighbour solution generated by perturbing one random gene using a partner source

Algorithm Performance Summary Table (n = 50):

Algorithm	Setting	Best Utility	Avg Utility	Avg Iterations	Avg Time (ms)
BCGA	Setting 1 (M=25)	0.493452	0.515359	36.5	245.59
BCGA	Setting 2 (M=40)	0.471976	0.493569	68.8	876.99
BCGA	Setting 3 (M=60)	0.505564	0.516318	77.9	1157.31
RCGA	Setting 1 (M=25)	0.496105	0.527922	53.2	540.23
RCGA	Setting 2 (M=40)	0.498651	0.539498	85.8	1532.38
RCGA	Setting 3 (M=60)	0.525298	0.553801	99.8	2150.29
Hybrid GA	Setting 1 (M=25)	0.493452	0.509005	39.5	267.31
Hybrid GA	Setting 2 (M=40)	0.475909	0.487776	76.2	1126.15
Hybrid GA	Setting 3 (M=60)	0.507870	0.517610	60.2	779.18
PSO	Setting 1 (M=25)	0.493452	0.502533	32.4	336.12
PSO	Setting 2 (M=40)	0.479580	0.518378	36.0	783.35
PSO	Setting 3 (M=60)	0.504638	0.535602	44.0	429.95
TLBO	Setting 1 (M=25)	0.493452	0.493452	34.6	999.81
TLBO	Setting 2 (M=40)	0.471976	0.474761	52.0	1365.22
TLBO	Setting 3 (M=60)	0.461278	0.468612	69.2	1691.90
DE	Setting 1 (M=25)	0.493452	0.493586	51.8	813.47
DE	Setting 2 (M=40)	0.471976	0.479049	85.5	1147.15
DE	Setting 3 (M=60)	0.473658	0.497028	110.5	1457.09
ABC	Setting 1 (M=25)	0.493452	0.532061	30.4	374.23
ABC	Setting 2	0.512121	0.562927	28.9	535.03

	(M=40)				
ABC	Setting 3 (M=60)	0.558447	0.588904	34.4	581.63

6. Conclusion

The experiment proves that different meta-heuristic algorithms greatly affect the outcome of the process of optimizing blockchain configuration and its quality, particularly in case more validators have to be taken into account.

TLBO turned out to be the most efficient algorithm providing the lowest and most stable utility values for various configurations and numbers of transactions per block, while DE ranked second in terms of effectiveness and performance.

Other simple algorithms, such as BCGA and Hybrid GA were satisfactory when the number of settings was small, yet less efficient in case of large-scale problems. ABC was revealed to be the least efficient meta-heuristic approach for the given task. Moreover, the experiment proved that a greater number of transactions per block leads to more efficient performance of the system.